Laboratorio 5: Miniuniverso 2D

Programación II - UPM - Curso 2024/2025

En este lab vamos a desarrollar un programa para realizar una simulación en dos dimensiones de un conjunto de *cuerpos celestes* en movimiento, sometidos a la *ley de gravitación universal*.

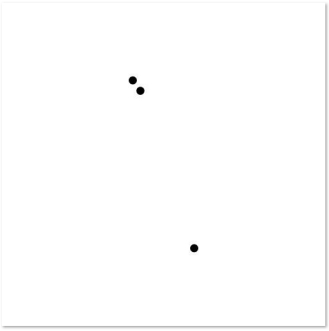
Vamos a usar arrays unidimensionales, lectura de datos del input mediante la clase `StdIn` y redireccionamiento de la entrada y gráficos mediante la clase `StdDraw`.

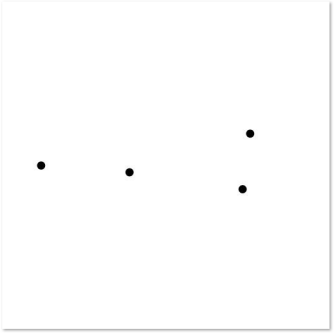
Se pide: Entregar los módulos `Vector`, `Planeta` y `Universo`.

El sitio web https://theskylive.com/ ofrece una simulación realista y muy ilustrativa del escenario celeste, así como una lista de los objetos celestes más conocidos.

El objetivo de este lab es desarrollar un programa para simular varios cuerpos en movimiento sometidos a la ley fundamental de la dinámica (https://es.wikipedia.org/wiki/Leyes_de_Newton#Segunda_ley_de_Newton_o_ley_fundamental_de_la_din%C3%A1mica). Dado un sistema de n cuerpos mutuamente afectados por las fuerzas gravitacionales, se trata de describir su movimiento. El mismo modelo básico se puede aplicar a problemas en un rango que va desde la escala astrofísica a la escala de la dinámica molecular.

En las dos imágenes siguientes se muestran dos simulaciones para dos escenarios diferentes, con tres y con cuatro objetos.





La clase `Vector`

La clase `Vector` nos va a ser muy útil para representar magnitudes vectoriales de la física mecánica, como la velocidad, el momento lineal, o la aceleración. Un vector es un objeto abstracto que tiene una *magnitud* y una *dirección*. La manera más habitual de representarlo gráficamente es mediante una flecha que va desde el origen hasta un punto cualquiera del sistema de coordenadas elegido. La dirección viene marcada por el ángulo que forma la flecha respecto a la horizontal, y la magnitud, por la longitud de la flecha. Para especificar el vector, basta con representar el punto.

Este concepto se extiende a cualquier número de dimensiones, de modo que para representar un vector de n dimensiones, vamos a utilizar un *array* de dimensión n.

Las operaciones que vamos a implementar sobre vectores serán las siguientes (https://es.wikipedia.org/wiki/Vector#Operaciones_con_vectores):

- Método `equals`: comparación por igualdad
- Método `sum`: suma de vectores
- Método `dif`: diferencia entre vectores
- Método `prod`: producto de un vector por un escalar
- Método `prod`: producto escalar
- Método `mod`: módulo de un vector
- Método `dir`: dirección de un vector
- Método `dist`: distancia entre vectores
- Método `dim`: dimensión de un vector
- Método `coord`: coordenada de un vector

En el siguiente texto de código Java tienes la especificación de `Vector`.

```
public class Vector {

  /** Dimensión del vector */
  private final int n;

  /** Array de coordenadas */
  private double[] coords;

  /**
   * Constructor: Crea un vector de dim n con coords a 0
   */
  public Vector (int n)

  /**
   * Constructor: Crea un vector a partir de un array
   *              Se copia el contenido del array
   */
  public Vector (double[] coords)

  /**
   * Constructor: Crea un vector desde una lista args
   *              El primer valor es la dimensión del vector
   */
  public Vector (String[] args)

  /**
   * POST: resultado: una representacion de este vector como texto
   */
  public String toString ()

  /**
   * POST: resultado: este Vector es igual a o
   */
  public boolean equals (Object o)

  /**
   * PRE: dim() = v.dim()
```

```
    * POST: resultado: this + v
    */
   public Vector sum (Vector v)

   /**
    * PRE: dim() = v.dim()
    * POST: resultado: this - v
    */
   public Vector dif (Vector v)

   /**
    * PRE: dim() = v.dim()
    * POST: resultado: el producto escalar de este Vector y v
    */
   public double prod (Vector v)

   /**
    * POST: resultado: k * this
    */
   public Vector prod (double k)

   /**
    * POST: resultado: modulo de este Vector
    * POST: resultado: |this|
    * POST: resultado: sqrt(this * this)
    */
   public double mod ()

   /**
    * PRE: mod() != 0
    * POST: resultado: vector unitario
    * POST: resultado: direccion de este vector
    * POST: resultado: this / |this|
    */
   public Vector dir ()

   /**
    * PRE: dim() = v.dim()
    * POST: resultado: distancia entre este Vector y v
    * POST: resultado: |this - v|
    */
   public double dist (Vector v)

   /** POST: resultado: dimensiones de este Vector
    */
   public double dim ()

   /**
    * POST: resultado: coordenada i de este Vector
    */
   public double coord (int i)

 }
```

Ejercicio 1: Implementar la clase `Vector`

VectorTest

El módulo `VectorTest` es un tester con diferentes pruebas de `Vector`. Utilízalo y añade otras pruebas.

```
import java.util.Arrays;

public class VectorTest {

  static void test1 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    Vector u = new Vector(uCoords);
    Assert.assertEquals(u.toString(), "(1.0,2.0,3.0,4.0)");
  }

  static void test2 ()
  {
    Vector u = new Vector(3);
    Assert.assertEquals(u.toString(), "(0.0,0.0,0.0)");
    u = new Vector(0);
    Assert.assertEquals(u.toString(), "()");
  }

  static void test3 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    double[] vCoords = { 5.0, 2.0, 4.0, 1.0 };
    Vector u = new Vector(uCoords);
    Vector v = new Vector(vCoords);
    Assert.assertEquals(u.sum(v).toString(), "(6.0,4.0,7.0,5.0)");
  }

  static void test4 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    double[] vCoords = { 5.0, 2.0, 4.0, 1.0 };
    Vector u = new Vector(uCoords);
    Vector v = new Vector(vCoords);
    Assert.assertEquals(u.dif(v).toString(), "(-4.0,0.0,-1.0,3.0)");
  }

  static void test5 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    double[] vCoords = { 5.0, 2.0, 4.0, 1.0 };
    Vector u = new Vector(uCoords);
    Vector v = new Vector(vCoords);
    Assert.assertEquals(u.prod(10.0).toString(), "(10.0,20.0,30.0,40.0)");
  }

  static void test6 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    double[] vCoords = { 5.0, 2.0, 4.0, 1.0 };
    Vector u = new Vector(uCoords);
    Vector v = new Vector(vCoords);
```

```
    Assert.assertEquals(u.prod(v), 25.0);
  }

  static void test7 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    Vector u = new Vector(uCoords);
    Assert.assertEquals(String.format("%.2f", u.mod()), "5,48");
  }

  static void test8 ()
  {
    double[] uCoords = { 2.0, 2.0, 2.0 };
    Vector u = new Vector(uCoords);
    Vector v = u.dir();
    String[] vRound = {String.format("%.2f",v.coord(0)),
                       String.format("%.2f",v.coord(1)),
                       String.format("%.2f",v.coord(2))};
    Assert.assertEquals(Arrays.toString(vRound), "[0,58, 0,58, 0,58]");
  }

  static void test9 ()
  {
    double[] uCoords = { 1.0, 2.0, 3.0, 4.0 };
    double[] vCoords = { 5.0, 2.0, 4.0, 1.0 };
    Vector u = new Vector(uCoords);
    Vector v = new Vector(vCoords);
    Assert.assertEquals(String.format("%.2f",u.dist(v)), "5,10");

    Assert.assertEquals(String.format("%.2f",u.dist(u)), "0,00");
  }

  public static void main(String[] args) {
    test1();
    test2();
    test3();
    test4();
    test5();
    test6();
    test7();
    test8();
    test9();
    System.out.println("Han pasado todas las pruebas");
  }
}
```

Ejercicio 2: Usar `VectorTest` para probar `Vector`

Una clase para modelar cuerpos en movimiento: `Cuerpo`

Vamos a representar un cuerpo en movimiento mediante dos vectores: `r`, con sus coordenadas y `v` con su velocidad. También tendremos un tercer atributo `m` que es un escalar con su masa.

Para calcular el desplazamiento de un cuerpo que está en `r`, que se mueve a una velocidad `v` durante un

tiempo `dt`, podemos escribir

```
r = r.sum(v.prod(dt));
```

La especificación de `Cuerpo` es la siguiente:

```
public class Cuerpo
{
  /** Posición del cuerpo celeste */
  private Vector r;

  /** Velocidad del cuerpo celeste */
  private Vector v;

  /** Masa del Cuerpo */
  private final double m;

  /**
   * Constructor: Crea un Cuerpo con posición, velocidad y masa
   */
  public Cuerpo (Vector r, Vector v, double m)

  /**
   * Mueve este Cuerpo aplicándole una fuerza f durante un tiempo dt
   *   a = f / m
   *   v += a * dt
   *   r += v * dt
   */
  public void mover (Vector f, double dt)

  /**
   * POST: resultado: fuerza ejercida por el Cuerpo c sobre este Cuerpo
   */
  public Vector fuerza (Cuerpo c)

  /**
   * POST: resultado: este Cuerpo como texto
   */
  public String toString ()

  /**
   * Dibuja este Cuerpo
   */
  public void dibujar () {
    StdDraw.setPenRadius(0.025);
    StdDraw.point(r.coord(0), r.coord(1));
  }

}
```

Ejercicio 3: Implementación de `Cuerpo`

`CuerpoTest`

Lo siguiente es un tester con diferentes pruebas de `Cuerpo`.

```
public class CuerpoTest {

  static void test1 ()
  {
    String linea = "0.0 4.5 1.0 0.0 1.5";
    Cuerpo c = Universo.cuerpo(linea);
    Assert.assertEquals(c.toString(), "(r=(0.0,4.5) v=(1.0,0.0) m=1.5)");

    linea = "0.0e00 4.5e10 1.0e04 0.0e00 1.5e30";
    c = Universo.cuerpo(linea);
    Assert.assertEquals(c.toString(),
                        "(r=(0.0,4.5E10) v=(10000.0,0.0) m=1.5E30)");
  }

  static void test2 ()
  {
    Cuerpo c = Universo.cuerpo("0.0 4.0 1.0 0.0 1.0");

    double[] fCoords = {2.0, 2.0};
    Vector f = new Vector(fCoords);
    double dt = 10;

    c.mover(f, dt);

    Assert.assertEquals(c.toString(),
                        "(r=(210.0,204.0) v=(21.0,20.0) m=1.0)");
  }

  static void test3 ()
  {
    Cuerpo c1 = Universo.cuerpo("0.0 4.0 1.0 0.0 1.0");
    Cuerpo c2 = Universo.cuerpo("0.0 6.0 2.0 0.0 2.0");
    Vector f = c1.fuerza(c2);

    Assert.assertEquals(String.format("%.3f",f.mod()/1e-11), "3,335");

    double[] coordsRes = {0.0, 1.0};
    Vector res = new Vector(coordsRes);
    Assert.assertEquals(f.dir(),res);
  }

  public static void main(String[] args) {
    test1();
    test2();
    test3();
    System.out.println("Han pasado todas las pruebas");
  }
}
```

Ejercicio 4: Usar `CuerpoTest` para probar `Cuerpo`

La clase `Universo`

La clase `Universo` es la encargada de manejar los n cuerpos en movimiento, haciendo que el estado del sistema avance en incrementos de tiempo `dt`. El método encargado de esto es `mover`. Este método crea un array con las n fuerzas que ejercen los n cuerpos (array de `Vector`(es)) entre sí, y a continuación mueve los n cuerpos aplicándoles estas fuerzas. Escrito en seudocódigo el algoritmo es:

```
mover (dt : double)
{
  f : Vector[], con longitud(f) = n
  crea e inicializa las fuerzas a cero en f
  calcula las fuerzas de interacción entre los n cuerpos en f
  mueve los n cuerpos aplicando las fuerzas en f durante el tiempo dt
}
```

La especificación de `Universo` se muestra continuación con algunos métodos implementados para facilitar la tarea.

```
public class Universo
{
  /** Constante de gravitación universal G */
  public static final double G = 6.67e-11;

  /** Tamaño del universo */
  private double radio;

  /** Número de cuerpos en este Universo*/
  private int n;

  /** Array de cuerpos */
  private Cuerpo[] cuerpos;

  /**
   * Constructor del Universo: se lee del input
   */
  public Universo () {
    n = 0;

    // leer el tamaño del universo
    radio = StdIn.readDouble();
    // salta a la linea siguiente
    StdIn.readLine();

    cuerpos = new Cuerpo[10];

    // leer los cuerpos
    while (!StdIn.isEmpty()) {
      String linea = StdIn.readLine();
      add(cuerpo(linea));
    }
  }

  /**
```

```
    * Crea un Cuerpo a partir de una línea de texto leída del input
    */
   public static Cuerpo cuerpo (String linea)

   /**
    * Añade el cuerpo c a este universo
    * Incrementa n
    */
   public void add (Cuerpo c)

   /**
    * PRE: las fuerzas son constantes en el tiempo dt
    *
    * Mueve el universo durante el tiempo dt.
    * Aplica las fuerzas de atracción entre los n cuerpos para que interactúen.
    */
   public void mover (double dt)

   /**
    * Dibuja los n cuerpos
    */
   public void dibujar ()

   /**
    * Convierte este universo a texto
    */
   public String toString ()

   /**
    * Simulación de este universo en movimiento con un incremento de tiempo dt
    */
   public void movimiento (double dt)
   {
     // la escala del dibujo:
     StdDraw.setXscale(-radio, +radio);
     StdDraw.setYscale(-radio, +radio);

     StdDraw.enableDoubleBuffering();
     while (true) {
       StdDraw.clear();
       mover(dt);
       dibujar();
       StdDraw.show();
       StdDraw.pause(10);
     }
   }

   public static void main (String[] args) {
     Universo u = new Universo();
     System.out.println(u);
     // Toma el incremento de tiempo dt de la línea de comandos:
     double dt = Double.parseDouble(args[0]);
     u.movimiento(dt);
   }
 }
```

Ejercicio 5: Implementar `Universo` y probarlo en modo gráfico

Puesta en marcha del `Universo`

Para poner en marcha la simulación, necesitamos cargar los datos de los objetos celestes. La manera más simple es mediante un archivo con el formato que se muestra en el siguiente ejemplo (archivo `2cuerpos.txt`):

```
5.0e10
0.0e00 4.5e10 1.0e04 0.0e00 1.5e30
0.0e00 -4.5e10 -1.0e04 0.0e00 1.5e30
```

La primera línea es el *radio* del espacio donde se desarrolla la simulación. Cada una de las siguientes líneas tiene los datos de *posición* y *velocidad* del objeto, con dos coordenadas para cada magnitud, por ser un espacio 2D. Entre cada valor tiene que haber exactamente un espacio.

La lectura de los datos es realizada por el constructor de `Universo`, que se da hecho. Cada objeto leído es añadido a la lista de objetos `cuerpos` mediante el método `add`.

Para poner en marcha la simulación, tenemos que pasar por la línea de comandos el *incremento de tiempo* de la simulación (`dt`) y redireccionar la entrada de datos. Si queremos un incremento de `25000` y tomar los datos del archivo anterior, escribiremos

```
> javac Universo.java
> java Universo 25000 < 2cuerpos.txt
```

Otros ejemplos de archivos de datos que crean simulaciones interesantes son:

- `3cuerpos.txt`

```
1.25e11
0.0e00 0.0e00 0.05e04 0.0e00 5.97e24
0.0e00 4.5e10 3.0e04 0.0e00 1.989e30
0.0e00 -4.5e10 -3.0e04 0.0e00 1.989e30
```

- `4cuerpos.txt`

```
5.0e10
-3.5e10 0.0e00 0.0e00 1.4e03 3.0e28
-1.0e10 0.0e00 0.0e00 1.4e04 3.0e28
1.0e10 0.0e00 0.0e00 -1.4e04 3.0e28
3.5e10 0.0e00 0.0e00 -1.4e03 3.0e28
```

- `2cuerposmini.txt`

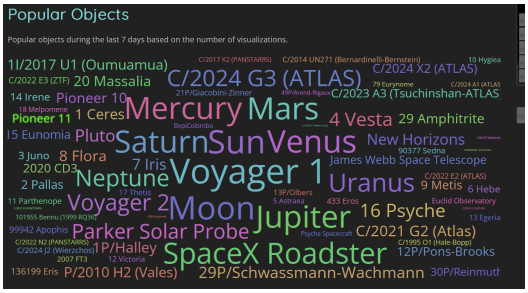
```
5.0e-10
0.0e00 4.5e-10 1.0e-16 0.0e00 1.5e-30
0.0e00 -4.5e-10 -1.0e-16 0.0e00 1.5e-30
```

Mejoras posibles

A partir de este desarrollo, podemos introducir distintas mejoras. Una línea de mejora podría ser introducir la característica de `Color` a la clase `Cuerpo` añadiendo un atributo de este tipo. Podemos añadir un constructor a `Cuerpo` que reciba un parámetro de tipo `Color` o que lo cree con un color aleatorio.

También se puede explorar la posibilidad de que el tamaño de la representación gráfica del cuerpo sea dependiente de su masa.

Otra línea de exploración es la de probar con variedades diferentes de objetos para generar simulaciones interesantes.



Ejercicio 6: Ampliar la clase `Cuerpo` y explorar distintos escenarios con diferentes datos